Snowflake Query Performance and Cost Optimization Guide

Objective

This document outlines best practices for analyzing query performance, improving large table efficiency using clustering keys, and monitoring warehouse cost usage in Snowflake.

---

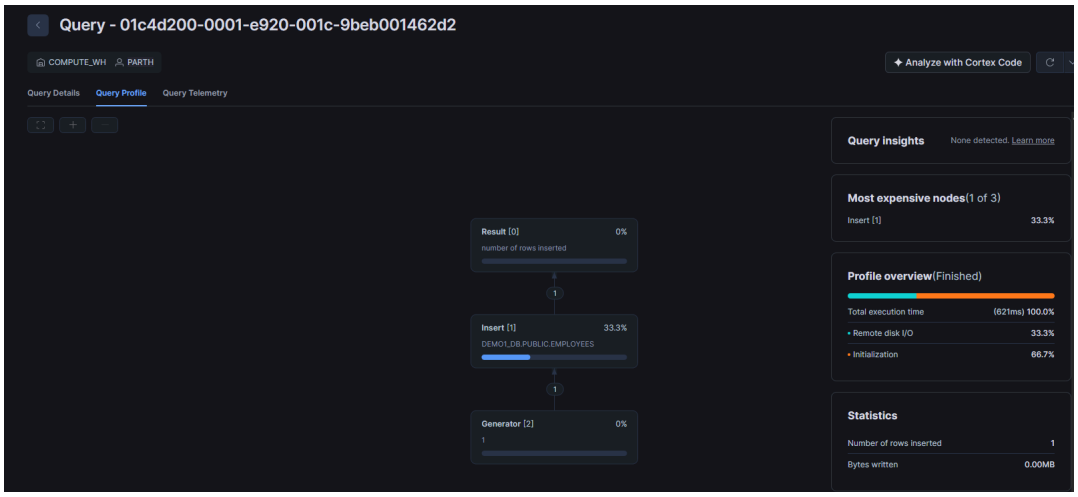
1. Analyze Query Performance Using Query Profiling

Purpose

Query Profiling helps identify performance bottlenecks and optimize query execution.

Step 1: Enable Query Profiling

1. Log in to the Snowflake Web UI.
2. Navigate to **Query History**.
3. Select a query from the history list.
4. Open the **Query Profile** tab to view execution details.



Step 2: Analyze the Query Profile

Review the profile and identify:

- High scan percentages
- Large intermediate result sets
- Expensive joins
- Long-running operations
- Inefficient filtering

Optimization Techniques

Add Filters

Reduce the amount of data scanned by applying appropriate filters.

```
SELECT *
FROM sales
WHERE sale_date >= '2025-01-01';
```

Use Proper Indexing Alternatives

Snowflake does not support traditional indexes. Instead:

- Use clustering keys for large tables.
- Organize data based on frequently queried columns.

Reduce Unnecessary Joins and Subqueries

Avoid joining tables that are not required for the analysis.

Before:

```
SELECT *
FROM orders o
JOIN customers c ON o.customer_id = c.customer_id
JOIN regions r ON c.region_id = r.region_id;
```

After (when region data is not needed):

```
SELECT *
FROM orders o
JOIN customers c ON o.customer_id = c.customer_id;
```

Expected Benefits

- Reduced query execution time
- Lower warehouse resource consumption
- Improved user experience

2. Experiment with Clustering Keys for Large Tables

Purpose

Clustering keys improve pruning efficiency and query performance on large tables.

Step 1: Create or Load a Large Dataset

Load a representative dataset into a table.

Example:

```
CREATE TABLE sales (
   sale_id NUMBER,
   sale_date DATE,
   customer_id NUMBER,
   amount NUMBER
);
```

Load data using Snowflake data loading methods.

Step 2: Add a Clustering Key

Choose a column that is frequently used in:

- WHERE clauses
- JOIN conditions
- Range filters

Example:

```
ALTER TABLE sales
```

ADD CLUSTERING KEY (sale_date);

Step 3: Monitor Clustering Performance

Review clustering effectiveness using:

SELECT SYSTEM$CLUSTERING_INFORMATION('sales');

Key Metrics to Review

- Clustering Depth
- Partition Overlap
- Clustering Quality

Best Practices

- Use clustering only for large tables.
- Select columns frequently used in filtering operations.
- Avoid excessive clustering keys.
- Periodically evaluate clustering effectiveness.

Expected Benefits

- Improved partition pruning
- Reduced data scanning
- Faster query execution

3. Monitor and Analyze Cost Usage Metrics

Purpose

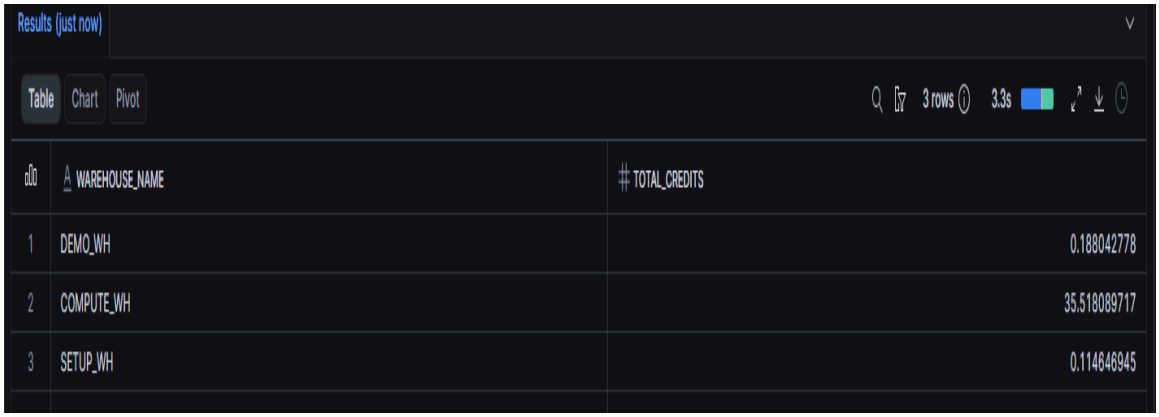
Monitoring warehouse consumption helps control Snowflake operational costs.

Step 1: Check Warehouse Usage

Retrieve warehouse credit consumption:

```
SELECT
   WAREHOUSE_NAME,
   SUM(CREDITS_USED) AS TOTAL_CREDITS
FROM SNOWFLAKE.ACCOUNT_USAGE.WAREHOUSE_METERING_HISTORY
WHERE START_TIME > CURRENT_TIMESTAMP() - INTERVAL '7 DAY'
GROUP BY WAREHOUSE_NAME;
```

Analyze Results



Step 2: Optimize Warehouse Costs

Suspend warehouses when they are not actively processing workloads.

```
ALTER WAREHOUSE my_warehouse SUSPEND;
```

Additional Cost Optimization Recommendations

Enable Auto-Suspend

```
ALTER WAREHOUSE my_warehouse
SET AUTO_SUSPEND = 60;
```

Enable Auto-Resume

```
ALTER WAREHOUSE my_warehouse
SET AUTO_RESUME = TRUE;
```

Right-Size Warehouses

Choose warehouse sizes based on workload requirements:

- X-Small for light workloads
- Small/Medium for standard workloads
- Large and above only for intensive processing

Summary

Optimization Area	Recommended Action	Benefit
Query Performance	Analyze Query Profile	Faster query execution
Data Organization	Implement Clustering Keys	Improved pruning and reduced scans
Cost Management	Monitor Warehouse Usage	Lower Snowflake credit consumption
Resource Optimization	Auto-Suspend Warehouses	Reduced idle costs

Conclusion

Regular monitoring of query performance, strategic use of clustering keys, and proactive warehouse cost management are essential for maintaining an efficient and cost-effective Snowflake environment. Following these practices can significantly improve performance while reducing overall operational expenses.